

SEGUIMIENTO V

Sara Rondon, Natalia Ángel
Electrónica digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	3
6. Explicación de funciones principales	3
7. Comunicación (si aplica)	4
8. Interfaz de usuario (si aplica)	4
9. Manejo de errores y seguridad	4
10.Código fuente	5

1. Identificación del proyecto

- **Nombre del proyecto:** Sistema IoT de monitoreo de temperatura, humedad y movimiento
- **Integrantes del equipo:** Sara Rondon, Natalia Ángel
- **Asignatura:** Electrónica Digital II

2. Resumen

El proyecto consiste en el desarrollo de un sistema IoT basado en el microcontrolador ESP32, orientado al monitoreo de variables ambientales y de movimiento en tiempo real. El sistema permite medir temperatura y humedad mediante un sensor DHT, así como detectar el estado dinámico del sistema mediante un acelerómetro MPU6050.

Se implementa un servidor web embebido que permite visualizar los datos en tiempo real desde un navegador, así como un bot de Telegram que permite el monitoreo remoto y la interacción mediante comandos. El sistema también incluye un conjunto de alertas multimodales que combinan señales sonoras, visualización web y notificaciones remotas.

3. Descripción del hardware

El sistema está basado en el microcontrolador ESP32, encargado del procesamiento de datos y la comunicación del sistema.

Entradas

- Sensor DHT11/DHT22 (GPIO 4): medición de temperatura y humedad.
- Sensor MPU6050 (I2C GPIO 21 SDA, GPIO 22 SCL): medición de aceleración y detección de movimiento.
- Pulsador (GPIO 18): botón de pánico.

Procesamiento

- Lectura de sensores y cálculo de variables ambientales.
- Análisis de movimiento mediante magnitud del vector de aceleración.
- Evaluación de umbrales y generación de alertas.

Salidas

- Buzzer pasivo (GPIO 27): generación de alertas sonoras.
- Comunicación web mediante WiFi.
- Notificaciones remotas mediante bot de Telegram.

El sistema es alimentado mediante el puerto USB del ESP32.

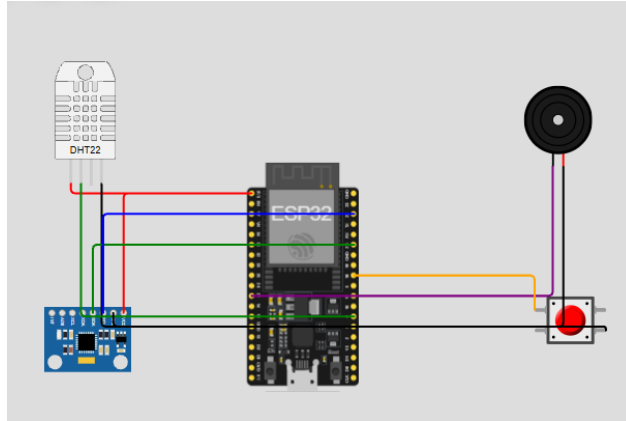


Figura 1: Diagrama de conexión del sistema

4. Descripción del software

El software está desarrollado en MicroPython y utiliza programación asíncrona mediante la librería `uasyncio`.

El sistema se divide en múltiples tareas concurrentes que permiten ejecutar simultáneamente la lectura de sensores, el servidor web y la comunicación con Telegram. Esto garantiza un funcionamiento eficiente y en tiempo real.

Se utilizan librerías como `network` para conexión WiFi, `urequests` para comunicación HTTP y `ujson` para manejo de datos.

5. Estructura del código

El código se encuentra organizado en los siguientes bloques:

- Configuración de parámetros globales y umbrales.
- Inicialización del hardware (sensores, buzzer y pulsador).
- Estado global del sistema.
- Funciones auxiliares.
- Sistema de alertas.
- Tareas asíncronas (sensores, servidor web y Telegram).
- Programa principal.

6. Explicación de funciones principales

- `obtener_ip()`: Obtiene la dirección IP del ESP32.

- `url_encode()`: Codifica mensajes para envío mediante URL.
- `enviar_telegram()`: Envía mensajes al bot de Telegram.
- `alerta_sonora()`: Genera diferentes patrones sonoros según el tipo de alerta.
- `disparar_alerta()`: Gestiona las alertas evitando envíos repetitivos.
- `leer_dht_seguro()`: Lee el sensor DHT manejando errores.
- `tarea_sensores()`: Gestiona la lectura de sensores y evaluación de condiciones.
- `servidor_web()`: Implementa el servidor HTTP embebido.
- `tarea_telegram()`: Gestiona la interacción con el bot de Telegram.

7. Comunicación (si aplica)

La comunicación del sistema se realiza mediante conexión WiFi.

El ESP32 ejecuta un servidor web embebido que permite la visualización de datos en tiempo real. Adicionalmente, se implementa un bot de Telegram que permite monitorear el sistema de forma remota y recibir alertas.

8. Interfaz de usuario (si aplica)

La interfaz de usuario está compuesta por dos elementos principales:

Interfaz web

- Visualización de temperatura, humedad y movimiento.
- Estado de alarmas.
- Actualización automática en tiempo real.

Bot de Telegram

- Permite consultar variables del sistema.
- Permite recibir alertas.
- Incluye comandos como `/estado`, `/temp`, `/humedad` y `/movimiento`.

9. Manejo de errores y seguridad

El sistema utiliza estructuras `try/except` para garantizar su estabilidad frente a errores de lectura de sensores o fallos de comunicación.

Además, se implementa control de spam en las alertas y liberación de memoria mediante `gc.collect()`, asegurando un funcionamiento continuo sin bloqueos.

10. Código fuente

```
1  import gc
2  import time
3  import ujson
4  import dht
5  import uasyncio as asyncio
6  import network
7  import urequests
8
9  from machine import Pin, I2C, PWM
10 from mpu6050 import MPU6050
11
12
13 # =====
14 # CONFIGURACION GENERAL
15 # =====
16
17 # IMPORTANTE:
18 TIPO_DHT = "DHT11"
19
20 # Configuración de Telegram.
21 # Cambiar por los datos reales del bot.
22 BOT_TOKEN = "8714231259:AAFgi3bfxKQXWop5iThKQC0tMV80P_EPDMM"
23 CHAT_ID = "8889804535"
24
25 # Pines ESP32.
26 PIN_DHT = 4
27 PIN_SDA = 21
28 PIN_SCL = 22
29 PIN_BUZZER = 27
30 PIN_PULSADOR = 18
31
32 # Umbrales ambientales.
33 TEMP_MIN = 18.0
34 TEMP_MAX = 35.0
35 HUM_MIN = 40.0
36 HUM_MAX = 70.0
37
38 # Umbrales de movimiento en unidades de g.
39 MOV_NORMAL_G = 1.15
40 MOV_BRUSCO_G = 1.80
41
42 # Intervalos de operación.
43 INTERVALO_SENSORES = 2          # segundos
44 INTERVALO_TELEGRAM = 5          # segundos
45 ANTI_SPAM_ALERTA = 60           # segundos entre alertas repetidas del mismo
    tipo
46
47
48 # =====
49 # INICIALIZACION DE HARDWARE
50 # =====
51
```

```

52 if TIPO_DHT.upper() == "DHT11":
53     sensor_dht = dht.DHT22(Pin(PIN_DHT))
54 else:
55     sensor_dht = dht.DHT11(Pin(PIN_DHT))
56
57 i2c = I2C(0, sda=Pin(PIN_SDA), scl=Pin(PIN_SCL), freq=400000)
58 mpu = MPU6050(i2c)
59
60 buzzer = PWM(Pin(PIN_BUZZER))
61 buzzer.duty(0)
62
63 pulsador = Pin(PIN_PULSADOR, Pin.IN, Pin.PULL_UP)
64
65
66 # =====
67 # ESTADO GLOBAL
68 # =====
69
70 estado = {
71     "temperatura": None,
72     "humedad": None,
73     "ax": 0.0,
74     "ay": 0.0,
75     "az": 0.0,
76     "magnitud_g": 1.0,
77     "movimiento": "SIN DATO",
78     "alarma": "INICIANDO",
79     "panico": "INACTIVO",
80     "ip": "SIN IP",
81     "dht_ok": False,
82     "ultima_actualizacion": "SIN DATO"
83 }
84
85 ultimo_envio_alerta = {}
86 telegram_offset = 0
87
88
89 # =====
90 # FUNCIONES AUXILIARES
91 # =====
92
93 def obtener_ip():
94     wlan = network.WLAN(network.STA_IF)
95     if wlan.isconnected():
96         return wlan.ifconfig()[0]
97     return "SIN CONEXION WIFI"
98
99
100 def url_encode(texto):
101     """
102     Codificacion basica para enviar texto por URL a Telegram.
103     Evita caracteres problematicos en MicroPython.
104     """
105     texto = str(texto)

```

```

106
107     reemplazos = {
108         "%": "%25",
109         " ": "%20",
110         "\n": "%0A",
111         ":": "%3A",
112         "/": "%2F",
113         "#": "%23",
114         "&": "%26",
115         "?": "%3F",
116         "=": "%3D",
117         "+": "%2B",
118         " ": "%C2%B0",
119         " ": "",
120         "!": "%21",
121         "a": "a", "e": "e", "i": "i", "o": "o", "u": "u",
122         "A": "A", "E": "E", "I": "I", "O": "O", "U": "U",
123         "n": "n", "N": "N"
124     }
125
126     for original, codificado in reemplazos.items():
127         texto = texto.replace(original, codificado)
128
129     return texto
130
131
132 def telegram_configurado():
133     return (
134         BOT_TOKEN != "TU_TOKEN_DE_TELEGRAM"
135         and CHAT_ID != "TU_CHAT_ID"
136         and len(BOT_TOKEN) > 10
137         and len(CHAT_ID) > 0
138     )
139
140
141 def mensaje_estado():
142     temp = "SIN LECTURA" if estado["temperatura"] is None else "{:.1f} C".
143         format(estado["temperatura"])
144     hum = "SIN LECTURA" if estado["humedad"] is None else "{:.1f} %".
145         format(estado["humedad"])
146
147     return (
148         "ESTADO DEL SISTEMA\n"
149         "Temperatura: {}\n"
150         "Humedad: {}\n"
151         "DHT: {}\n"
152         "Movimiento: {}\n"
153         "Magnitud: {:.2f} g\n"
154         "Alarma: {}\n"
155         "Boton panico: {}\n"
156         "IP: {}"
157     ).format(
158         temp,
159         hum,

```

```

158         "OK" if estado["dht_ok"] else "SIN LECTURA",
159         estado["movimiento"],
160         estado["magnitud_g"],
161         estado["alarma"],
162         estado["panico"],
163         estado["ip"]
164     )
165
166
167 def mensaje_umbrales():
168     return (
169         "UMBRALES CONFIGURADOS\n"
170         "Temperatura minima: {:.1f} C\n"
171         "Temperatura maxima: {:.1f} C\n"
172         "Humedad minima: {:.1f} %\n"
173         "Humedad maxima: {:.1f} %\n"
174         "Movimiento normal: > {:.2f} g\n"
175         "Movimiento brusco: > {:.2f} g"
176     ).format(TEMP_MIN, TEMP_MAX, HUM_MIN, HUM_MAX, MOV_NORMAL_G,
177             MOV_BRUSCO_G)
178
179 async def enviar_telegram(mensaje):
180     """
181     Envia mensaje por Telegram.
182     Si BOT_TOKEN o CHAT_ID no estan configurados, solo imprime en consola.
183     """
184     if not telegram_configurado():
185         print("Telegram no configurado. Mensaje local:", mensaje)
186         return False
187
188     try:
189         url = (
190             "https://api.telegram.org/bot{}/sendMessage?chat_id={}&text"
191             "={}"
192             .format(BOT_TOKEN, CHAT_ID, url_encode(mensaje))
193         )
194
195         respuesta = urequests.get(url)
196         codigo = respuesta.status_code
197         respuesta.close()
198
199         if codigo == 200:
200             print("Mensaje enviado a Telegram.")
201             return True
202         else:
203             print("Telegram respondio con codigo:", codigo)
204             return False
205
206     except Exception as e:
207         print("Error enviando Telegram:", e)
208         return False
209

```



```

210 async def alerta_sonora(tipo):
211     """
212     Patrones sonoros diferenciados.
213     """
214     if tipo == "TEMPERATURA":
215         frecuencia = 1000
216         repeticiones = 2
217     elif tipo == "HUMEDAD":
218         frecuencia = 1300
219         repeticiones = 3
220     elif tipo == "MOVIMIENTO":
221         frecuencia = 2000
222         repeticiones = 4
223     elif tipo == "PANICO":
224         frecuencia = 2500
225         repeticiones = 5
226     elif tipo == "DHT":
227         frecuencia = 700
228         repeticiones = 1
229     else:
230         frecuencia = 900
231         repeticiones = 1
232
233     for _ in range(repeticiones):
234         buzzer.freq(frecuencia)
235         buzzer.duty(512)
236         await asyncio.sleep(0.16)
237         buzzer.duty(0)
238         await asyncio.sleep(0.12)
239
240     buzzer.duty(0)
241
242
243 async def disparar_alerta(tipo, mensaje):
244     """
245     Activa alerta sonora y alerta remota evitando spam.
246     """
247     ahora = time.time()
248     ultimo = ultimo_envio_alerta.get(tipo, 0)
249
250     await alerta_sonora(tipo)
251
252     if ahora - ultimo >= ANTI_SPAM_ALERTA:
253         ultimo_envio_alerta[tipo] = ahora
254         await enviar_telegram("ALERTA {}:\\n{}".format(tipo, mensaje))
255
256     print("ALERTA", tipo, mensaje)
257
258
259 def leer_dht_seguro():
260     """
261     Lee el DHT evitando que ETIMEDOUT detenga todo el sistema.
262     Si falla, mantiene los valores anteriores y marca dht_ok = False.
263     """

```

```

264     try:
265         sensor_dht.measure()
266         temperatura = float(sensor_dht.temperature())
267         humedad = float(sensor_dht.humidity())
268
269         estado["temperatura"] = temperatura
270         estado["humedad"] = humedad
271         estado["dht_ok"] = True
272
273         return temperatura, humedad, True
274
275     except Exception as e:
276         print("Error leyendo DHT:", e)
277         estado["dht_ok"] = False
278
279         # Mantiene ultimos valores validos.
280         return estado["temperatura"], estado["humedad"], False
281
282
283 # =====
284 # TAREA 1: MONITOREO DE SENSORES
285 # =====
286
287 async def tarea_sensores():
288     estado["ip"] = obtener_ip()
289
290     await enviar_telegram(
291         "ESP32 iniciado correctamente.\nIP del servidor web: http://{ }".
292         format(estado["ip"])
293     )
294
295     while True:
296         alarmas = []
297
298         try:
299             # Lectura DHT protegida.
300             temperatura, humedad, dht_ok = leer_dht_seguro()
301
302             # Lectura MPU6050 protegida.
303             try:
304                 ax, ay, az = mpu.leer_aceleracion_g()
305                 magnitud = mpu.leer_magnitud_g()
306                 movimiento = mpu.clasificar_movimiento(magnitud)
307
308                 estado["ax"] = ax
309                 estado["ay"] = ay
310                 estado["az"] = az
311                 estado["magnitud_g"] = magnitud
312                 estado["movimiento"] = movimiento
313
314             except Exception as e:
315                 print("Error leyendo MPU6050:", e)
316                 estado["movimiento"] = "ERROR MPU6050"
317                 magnitud = estado["magnitud_g"]

```

```

317         movimiento = estado["movimiento"]
318
319     estado["panico"] = "ACTIVO" if pulsador.value() == 0 else "
        INACTIVO"
320     estado["ultima_actualizacion"] = str(time.time())
321     estado["ip"] = obtener_ip()
322
323     # Solo evalua temperatura y humedad si el DHT tiene lectura
        valida.
324     if dht_ok:
325         if temperatura < TEMP_MIN:
326             alarmas.append("TEMPERATURA BAJA")
327             await disparar_alerta(
328                 "TEMPERATURA",
329                 "Temperatura baja: {:.1f} C. Limite minimo: {:.1f}
                    C"
330                 .format(temperatura, TEMP_MIN)
331             )
332
333         if temperatura > TEMP_MAX:
334             alarmas.append("TEMPERATURA ALTA")
335             await disparar_alerta(
336                 "TEMPERATURA",
337                 "Temperatura alta: {:.1f} C. Limite maximo: {:.1f}
                    C"
338                 .format(temperatura, TEMP_MAX)
339             )
340
341         if humedad < HUM_MIN:
342             alarmas.append("HUMEDAD BAJA")
343             await disparar_alerta(
344                 "HUMEDAD",
345                 "Humedad baja: {:.1f} %. Limite minimo: {:.1f} %"
346                 .format(humedad, HUM_MIN)
347             )
348
349         if humedad > HUM_MAX:
350             alarmas.append("HUMEDAD ALTA")
351             await disparar_alerta(
352                 "HUMEDAD",
353                 "Humedad alta: {:.1f} %. Limite maximo: {:.1f} %"
354                 .format(humedad, HUM_MAX)
355             )
356     else:
357         alarmas.append("DHT SIN LECTURA")
358         await disparar_alerta(
359             "DHT",
360             "No se pudo leer el sensor DHT. Revise tipo de sensor,
                DATA GPIO 4, VCC 3V3 y GND."
361         )
362
363     if estado["movimiento"] == "MOVIMIENTO BRUSCO":
364         alarmas.append("MOVIMIENTO BRUSCO")
365         await disparar_alerta(

```

```

366         "MOVIMIENTO",
367         "Movimiento brusco detectado. Magnitud: {:.2f} g"
368         .format(estado["magnitud_g"])
369     )
370
371     if pulsador.value() == 0:
372         alarmas.append("BOTON DE PANICO")
373         await disparar_alerta(
374             "PANICO",
375             "Boton de panico activado manualmente."
376         )
377
378     if len(alarmas) == 0:
379         estado["alarma"] = "NORMAL"
380         buzzer.duty(0)
381     else:
382         estado["alarma"] = " / ".join(alarmas)
383
384     except Exception as e:
385         estado["alarma"] = "ERROR GENERAL"
386         buzzer.duty(0)
387         print("Error general en tarea_sensores:", e)
388
389     gc.collect()
390     await asyncio.sleep(INTERVALO_SENSORES)
391
392
393 # =====
394 # TAREA 2: SERVIDOR WEB
395 # =====
396
397 def pagina_web():
398     color_alarma = "#0f9d58" if estado["alarma"] == "NORMAL" else "#d93025"
399
400     temp_web = "SIN LECTURA" if estado["temperatura"] is None else "{:.1f}
401     }.format(estado["temperatura"])
402     hum_web = "SIN LECTURA" if estado["humedad"] is None else "{:.1f}
403     }.format(estado["humedad"])
404
405     html = """<!DOCTYPE html>
406     <html>
407     <head>
408         <meta charset="UTF-8">
409         <meta http-equiv="refresh" content="3">
410         <title>Seguimiento V - ESP32</title>
411         <style>
412             body {
413                 font-family: Arial, sans-serif;
414                 background: #f3f6fb;
415                 margin: 0;
416                 padding: 24px;
417                 color: #1f2933;
418             }

```

```

417     .card {{
418         max-width: 760px;
419         margin: auto;
420         background: white;
421         border-radius: 14px;
422         padding: 24px;
423         box-shadow: 0 4px 18px rgba(0,0,0,0.12);
424     }}
425     h1 {{
426         margin-top: 0;
427         color: #0b5394;
428     }}
429     .grid {{
430         display: grid;
431         grid-template-columns: repeat(2, 1fr);
432         gap: 14px;
433     }}
434     .box {{
435         background: #eef3f8;
436         border-radius: 10px;
437         padding: 14px;
438     }}
439     .value {{
440         font-size: 28px;
441         font-weight: bold;
442     }}
443     .alarm {{
444         background: {color_alarm};
445         color: white;
446         border-radius: 10px;
447         padding: 14px;
448         font-size: 20px;
449         font-weight: bold;
450         margin: 16px 0;
451     }}
452     .small {{
453         font-size: 13px;
454         color: #5f6b7a;
455     }}
456     table {{
457         width: 100%;
458         border-collapse: collapse;
459         margin-top: 12px;
460     }}
461     td, th {{
462         padding: 8px;
463         border-bottom: 1px solid #d8dee9;
464         text-align: left;
465     }}
466 </style>
467 </head>
468 <body>
469     <div class="card">
470         <h1>Monitoreo IoT Biomedico - ESP32</h1>

```

```

471     <p class="small">Servidor web embebido con actualizacion
        automatica cada 3 segundos.</p>
472
473     <div class="alarm">Estado de alarma: {alarma}</div>
474
475     <div class="grid">
476         <div class="box">
477             <div>Temperatura</div>
478             <div class="value">{temperatura} C</div>
479         </div>
480         <div class="box">
481             <div>Humedad relativa</div>
482             <div class="value">{humedad} %</div>
483         </div>
484         <div class="box">
485             <div>Estado de movimiento</div>
486             <div class="value">{movimiento}</div>
487         </div>
488         <div class="box">
489             <div>Magnitud aceleracion</div>
490             <div class="value">{magnitud:.2f} g</div>
491         </div>
492     </div>
493
494     <h2>Estado del sistema</h2>
495     <table>
496         <tr><th>Variable</th><th>Valor</th></tr>
497         <tr><td>Sensor DHT</td><td>{dht_estado}</td></tr>
498         <tr><td>Boton de panico</td><td>{panico}</td></tr>
499         <tr><td>Aceleracion X</td><td>{ax:.2f} g</td></tr>
500         <tr><td>Aceleracion Y</td><td>{ay:.2f} g</td></tr>
501         <tr><td>Aceleracion Z</td><td>{az:.2f} g</td></tr>
502         <tr><td>IP ESP32</td><td>{ip}</td></tr>
503     </table>
504
505     <h2>Umbrales configurados</h2>
506     <table>
507         <tr><th>Parametro</th><th>Rango / limite</th></tr>
508         <tr><td>Temperatura</td><td>{tmin:.1f} C a {tmax:.1f} C</td></tr>
509         <tr><td>Humedad</td><td>{hmin:.1f} % a {hmax:.1f} %</td></tr>
510         <tr><td>Movimiento</td><td>Normal &gt; {mov:.2f} g</td></tr>
511         <tr><td>Movimiento brusco</td><td>&gt; {brusco:.2f} g</td></tr>
512     </table>
513 </div>
514 </body>
515 </html>
516 """.format(
517     color_alarma=color_alarma,
518     alarma=estado["alarma"],
519     temperatura=temp_web,
520     humedad=hum_web,
521     movimiento=estado["movimiento"],

```

```

522     magnitud=estado["magnitud_g"],
523     dht_estado="OK" if estado["dht_ok"] else "SIN LECTURA",
524     panico=estado["panico"],
525     ax=estado["ax"],
526     ay=estado["ay"],
527     az=estado["az"],
528     ip=estado["ip"],
529     tmin=TEMP_MIN,
530     tmax=TEMP_MAX,
531     hmin=HUM_MIN,
532     hmax=HUM_MAX,
533     mov=MOV_NORMAL_G,
534     brusco=MOV_BRUSCO_G
535 )
536
537 return html
538
539
540 async def servidor_web():
541     async def atender_cliente(reader, writer):
542         try:
543             request = await reader.read(1024)
544             request = request.decode()
545
546             if "GET /json" in request:
547                 contenido = ujson.dumps(estado)
548                 writer.write("HTTP/1.1 200 OK\r\n")
549                 writer.write("Content-Type: application/json\r\n")
550                 writer.write("Connection: close\r\n\r\n")
551                 writer.write(contenido)
552             else:
553                 contenido = pagina_web()
554                 writer.write("HTTP/1.1 200 OK\r\n")
555                 writer.write("Content-Type: text/html\r\n")
556                 writer.write("Connection: close\r\n\r\n")
557                 writer.write(contenido)
558
559             await writer.drain()
560             await writer.wait_closed()
561
562         except Exception as e:
563             print("Error cliente web:", e)
564
565     await asyncio.start_server(atender_cliente, "0.0.0.0", 80)
566     print("Servidor web iniciado en http://{}/".format(obtener_ip()))
567
568     while True:
569         await asyncio.sleep(3600)
570
571
572 # =====
573 # TAREA 3: BOT DE TELEGRAM
574 # =====
575

```

```

576 async def tarea_telegram():
577     global telegram_offset
578
579     if not telegram_configurado():
580         print("Telegram no configurado. Configure BOT_TOKEN y CHAT_ID para
          activar el bot.")
581         return
582
583     while True:
584         try:
585             url = (
586                 "https://api.telegram.org/bot{}/getUpdates?offset={}"
587                 .format(BOT_TOKEN, telegram_offset)
588             )
589
590             respuesta = urequests.get(url)
591             datos = respuesta.json()
592             respuesta.close()
593
594             if datos.get("ok"):
595                 for item in datos.get("result", []):
596                     telegram_offset = item["update_id"] + 1
597
598                     mensaje = item.get("message", {})
599                     texto = mensaje.get("text", "").strip().lower()
600
601                     if texto == "/estado":
602                         await enviar_telegram(mensaje_estado())
603
604                     elif texto == "/temp":
605                         if estado["temperatura"] is None:
606                             await enviar_telegram("Temperatura: SIN
          LECTURA")
607                         else:
608                             await enviar_telegram(
609                                 "Temperatura actual: {:.1f} C".format(
610                                     estado["temperatura"])
611                             )
612
613                     elif texto == "/humedad":
614                         if estado["humedad"] is None:
615                             await enviar_telegram("Humedad: SIN LECTURA")
616                         else:
617                             await enviar_telegram(
618                                 "Humedad actual: {:.1f} %".format(estado["
          humedad"])
619                             )
620
621                     elif texto == "/movimiento":
622                         await enviar_telegram(
623                             "Movimiento: {} \n Magnitud: {:.2f} g"
624                             .format(estado["movimiento"], estado["
          magnitud_g"])

```



```

625         elif texto == "/umbrales":
626             await enviar_telegram(mensaje_umbrales())
627
628         elif texto == "/ayuda":
629             await enviar_telegram(
630                 "Comandos disponibles:\n"
631                 "/estado\n"
632                 "/temp\n"
633                 "/humedad\n"
634                 "/movimiento\n"
635                 "/umbrales\n"
636                 "/ayuda"
637             )
638
639         elif texto:
640             await enviar_telegram("Comando no reconocido. Use
641                                   /ayuda")
642
643     except Exception as e:
644         print("Error revisando Telegram:", e)
645
646     gc.collect()
647     await asyncio.sleep(INTERVALO_TELEGRAM)
648
649 # =====
650 # PROGRAMA PRINCIPAL
651 # =====
652
653
654 async def main():
655     print("Iniciando sistema Seguimiento V...")
656     estado["ip"] = obtener_ip()
657
658     asyncio.create_task(tarea_sensores())
659     asyncio.create_task(servidor_web())
660     asyncio.create_task(tarea_telegram())
661
662     while True:
663         await asyncio.sleep(10)
664
665
666 try:
667     asyncio.run(main())
668 finally:
669     buzzer.duty(0)
670     asyncio.new_event_loop()

```